WEEK 1 — DAY 1

Optical Camera Communication (OCC) Video Message Decoding

Problem Understanding & System Definition

➢ Problem Statement

Optical Camera Communication uses a standard video camera as the receiver for a light-based data link. A transmitter — an LED or similar light source, is switched on and off rapidly to encode a binary message using On-Off Keying (OOK). Because the transmitter's blink rate is often faster than the camera's frame rate, the rolling-shutter exposure of each frame can capture multiple light/dark transitions as visible stripes within a single image, rather than one clean on/off value per frame.

My task is to build a software-only, offline pipeline that takes the raw OCC video files as input and reconstructs the transmitted
binary sequence — and from it, the decoded message — with no physical hardware, live camera access, or embedded deployment involved. The work is entirely dataset-driven: all videos are supplied by the faculty in advance.

➢ Inputs and Required Output

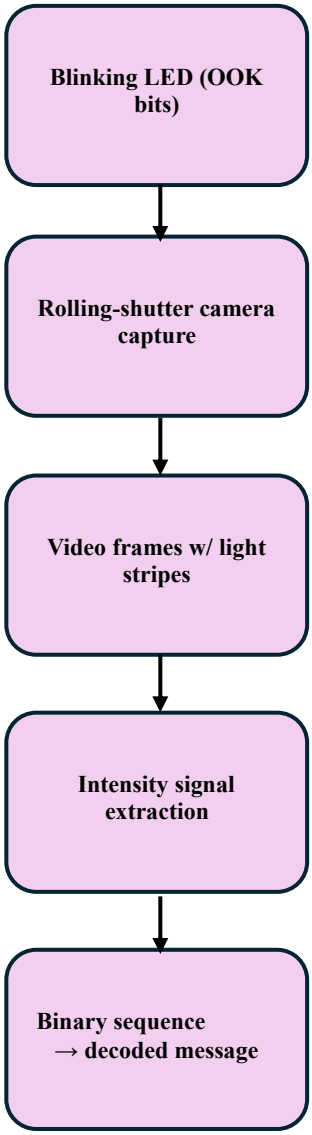
Inputs:

- Raw OCC video files, one or more per transmitted message, containing OOK-encoded light intensity variation.
- Associated metadata where available: frame rate, resolution, exposure/shutter type, capture distance/conditions.
- Ground-truth labels where available: transmitted bit sequence and/or decoded message text, for evaluation.

Required Output / Target:

- The transmitted binary sequence recovered from the video's light-intensity variation.
- The final decoded message reconstructed from that binary sequence.

➢ End-to-End Software Pipeline



Each stage is described further below:

- **Ingest & audit:** load raw videos, verify fps, resolution, duration, and frame integrity; preserve an untouched raw copy.
- **Locate transmitter (ROI):** detect or track the light source across frames to define the region of interest.
- **Extract intensity signal:** aggregate pixel intensity within the ROI per frame (or per rolling-shutter row) into a 1-D time series.
- **Preprocess signal:** denoise, normalize, and smooth to counter ambient flicker and exposure drift.
- **Synchronize symbols:** recover bit timing/clock so each intensity sample maps to one transmitted symbol.
- **Decode bits:** apply a fixed/adaptive threshold, signal-processing rule, or ML classifier to output 0/1 symbols.
- **Reconstruct message:** group bits into the message format and validate against expected structure (checksums/framing if present).

- **Evaluate & iterate:** score decoded-message correctness, BER (where reference truth exists), synchronization robustness, and execution time; refine upstream stages based on failures.

➢ Success Criteria

- The pipeline correctly reconstructs the transmitted message (or bit sequence) for videos with usable signal quality.
- Decoding performance is measured objectively: decoded-message correctness, bit-error rate where ground truth is available, synchronization robustness, and execution time.
- The full pipeline — from raw video to decoded output — is reproducible end-to-end by an independent reviewer.
- Failure cases are identified, documented, and explained rather than silently dropped.

➢ Technical Questions & Assumptions Requiring Clarification

- What is the expected symbol/bit rate (blinks per second) of the transmitter, and is it constant across all provided videos?
- Is ground-truth bit sequence / decoded message text available for all videos, or only a subset — and how should unlabelled videos be evaluated?
- Is there a fixed message framing structure (e.g., preamble, header, checksum) I should assume for message reconstruction, or should this be inferred?
- Are all videos captured with a static camera and static transmitter, or should I assume possible camera motion / handheld capture?
- What is the expected range of camera frame rates and resolutions across the dataset — is it a single camera/setup or multiple?
- Should ambient lighting conditions (e.g., indoor/outdoor, flicker from mains-powered lights) be treated as a variable to test robustness against, or are all videos captured under controlled lighting?
- Is exact bit-for-bit correctness required, or is a bit-error-rate threshold considered an acceptable decode for evaluation purposes?
- Should the pipeline assume one message per video, or could a single video contain a repeated/looping transmission that should be decoded from any complete cycle?

➢ Next Step

- With problem framing, inputs/outputs, pipeline design, and success criteria defined, the next step (Day 2) is a structured literature review of prior OCC decoding approaches, followed by dataset inspection once the faculty-provided videos are available.